BTS SIO — MINIPROJET HÉRITAGE EN ÉQUIPE (PHP ET C#)

Cahier des Charges Technique : Arme & Épée

> Voici votre cahier des charges technique. Suivez exactement ces noms et ces règles chacun de votre côté : vos projets PHP et C# seront strictement identiques et valideront 100 % des consignes.

1. Environnement & Base de Données (En commun)

- **GitHub :** Créer le dépôt avec 4 dossiers (`/SQL` , `/UML` , `/PHP` , `/CSharp`).
- **Présentation :** *"Système de gestion d'inventaire pour un jeu d'action permettant de gérer des armes génériques et des épées avec calcul dynamique de leur puissance."*
- **UML :** Dessiner la classe `Epee` qui hérite (flèche triangle vide) de la classe `Arme` . Attributs en privé (`-`), méthodes en public (`+`).
- **Tables MySQL :**
  - Table `arme` : `id` (INT, PK, Auto-Increment), `nom` (VARCHAR 50), `degatsBase` (INT), `poids` (DOUBLE).
  - Table `epee` : `id` (INT, PK et FK vers `arme.id`), `longueurLame` (DOUBLE), `tranchant` (INT).

2. Spécifications des Classes Métiers (Contrat strict PHP / C#)

Ton binôme C# doit créer un projet **Bibliothèque de classes (.dll)** sous Visual Studio. Toi, tu crées tes fichiers sous VSCode. Vous devez tous les deux organiser votre code avec ces 5 régions (`#region` en C#, commentaires en PHP) : *Champs privés, Constructeur, Accesseurs/Propriétés, Méthodes, ToString*.

Élément	Classe Mère : `Arme`	Classe Fille : `Epee` (hérite de `Arme`)
Champs privés	`id` (int), `nom` (string), `degatsBase` (int), `poids` (float/double)	`longueurLame` (float/double), `tranchant` (int)
Constructeur	`Arme(id, nom, degatsBase, poids)` *Doit utiliser les setteurs pour activer les exceptions.*	`Epee(id, nom, degatsBase, poids, longueurLame, tranchant)` *Appelle le constructeur parent (`parent::__construct` / `: base`) avec les 4 premiers paramètres, puis utilise ses setteurs pour les 2 derniers.*
Getters / Setters	Tous les 4 champs en lecture/écriture.	Les 2 champs spécifiques en lecture/écriture.

Élément	Classe Mère : `Arme`	Classe Fille : `Epee` (hérite de `Arme`)
Exceptions (dans les Setters)	• Si `degatsBase <= 0` : lever Exception *"Les dégâts doivent être positifs."* • Si `poids <= 0` : lever Exception *"Le poids doit être positif."*	• Si `longueurLame <= 0` : lever Exception *"La longueur de lame doit être positive."* • Si `tranchant < 0` : lever Exception *"Le tranchant ne peut pas être négatif."*
Méthode métier (Polymorphisme)	`calculerPuissance()` *(C# : `virtual`)* Retourne : `degatsBase / poids`	`calculerPuissance()` *(C# : `override`)* Retourne : `calculerPuissance() de la mère + tranchant`
Affichage (`toString`)	Retourne : `"[id] nom - Dégâts: degatsBase, Poids: poids kg"`	Retourne : `toString() de la mère + " | Épée (Lame: longueurLame cm, Tranchant: tranchant)"`

3. Scénario du Programme de Test (Identique PHP / C#)

Ton binôme crée un projet **Application Console** (qui référence sa DLL). Tu crées un fichier `test.php` (qui inclut tes classes). Vous codez exactement ces 3 étapes dans le même ordre :

1. **Instanciation et Collection :**
   - Créer une collection typée `Arme` (PHP : `$inventaire = []` / C# : `List<Arme> inventaire = new List<Arme>()`).
   - Ajouter 1 `Arme` valide : `(1, "Gourdin", 15, 3.0)`.
   - Ajouter 2 `Epee` valides : `(2, "Excalibur", 50, 2.5, 90.0, 25)` et `(3, "Katana", 40, 1.5, 75.0, 35)`.
2. **Boucle Polymorphe :**
   - Faire un `foreach` sur `inventaire`.
   - À chaque tour, afficher l'objet (`toString`) ET le résultat de `calculerPuissance()`. *(Prouve que C# et PHP exécutent automatiquement la formule de l'épée quand l'objet est une épée).*
3. **Test des Exceptions (Try / Catch) :**
   - Ouvrir un bloc `try`.
   - Tenter de créer une `Epee` invalide avec une lame négative : `(4, "Épée brisée", 10, 1.0, -50.0, 5)`.
   - Dans le bloc `catch (Exception e)`, afficher le message de l'exception capturée à l'écran.

4. Livrables du Rapport Pronote (À assembler à la fin)

1. **Lien GitHub** + les 2 phrases de présentation du domaine fonctionnel.
2. **Diagramme UML** (image).
3. **Base de données :** Code SQL complet + captures d'écran des 2 tables créées.
4. **Code source des classes :** Copier-coller de `Arme.php`, `Epee.php`, `Arme.cs`, `Epee.cs`.
5. **Tests :** Copier-coller des 2 fichiers de test + captures d'écran des résultats d'exécution (navigateur/terminal pour PHP, console pour C#) montrant l'affichage de la liste polymorphe et le message d'erreur du `try/catch`.

Vérification du Rendu Pronote

- **Lien vers le dépôt GitHub :** Prévu dans la Partie 1 et 4.
- **Présentation rapide du domaine fonctionnel :** La phrase prête à copier-coller est fournie dans la Partie 1.
- **Diagramme de classes (UML) :** Prévu dans la Partie 1 et 4.
- **Copies d'écran des tables + code SQL :** Prévu dans la Partie 1 et 4.
- **Code source complet des classes (PHP et C#) :** Les 4 fichiers (`Arme.php`, `Epee.php`, `Arme.cs`, `Epee.cs`) prévus en Partie 2 et 4.
- **Code et captures d'écran des tests (incluant l'exception) :** Scénario en 3 étapes prévu en Partie 3, avec captures du rendu final en Partie 4.

3 petits détails de vigilance pour avoir la note maximale :

1. **En C# (pour ton binôme) :** L'énoncé demande à la fois *"propriétés"* et *"accesseurs (getteurs/setteurs)"*. Dis bien à ton ami d'écrire des propriétés complètes avec `get` et `set` explicites (et non des propriétés automatiques `{ get; set; }`), ou de faire à la fois les méthodes `Get.../Set...` et les propriétés, selon l'habitude de votre professeur, pour pouvoir y loger les `if` des exceptions.
2. **En SQL (mapping strict) :** Pour que le professeur voie immédiatement que les champs sont mappés à l'identique, nommez la clé primaire de la table `epee` simplement `id` (qui sera à la fois `PRIMARY KEY` et `FOREIGN KEY REFERENCES arme(id)`), exactement comme l'attribut `$id` de vos classes.
3. **En UML :** N'oubliez pas de faire figurer les constructeurs, les getters/setters, `calculerPuissance()` et `toString()` dans la partie basse des boîtes de vos deux classes sur le diagramme.